

Corporación Universitaria Iberoamericana

Programa de Desarrollo de Software

Actividad 2

Búsqueda y sistemas basados en reglas

Sistema inteligente para la búsqueda de rutas en TransMilenio

Presentado por

Nelson Andrés Ramírez Gutiérrez

Curso: Inteligencia artificial

Docente: Sandra Bautista

Código del curso: 24082026_C1_202634

Bogotá, Colombia

2026

Introducción

Los sistemas basados en reglas representan conocimiento mediante hechos y condiciones que permiten inferir acciones. En este proyecto se combinan dichas reglas con un grafo y el algoritmo A* para recomendar una ruta entre dos estaciones de un modelo simplificado de TransMilenio. El caso facilita observar cómo la representación del conocimiento condiciona una búsqueda y cómo el sistema responde ante cierres o entradas inválidas.

El modelo tiene fines académicos. Sus costos y posiciones son estimaciones didácticas y no deben utilizarse para planear viajes reales.

Planteamiento del problema

¿Cómo construir en Python un sistema inteligente que utilice reglas lógicas y búsqueda heurística para obtener una ruta válida y de costo mínimo entre dos estaciones?

Objetivo general

Desarrollar un sistema en Python que represente una red de transporte mediante una base de conocimiento y aplique reglas lógicas y A* para seleccionar una ruta entre un origen y un destino.

Objetivos específicos

1. Modelar estaciones, conexiones y costos como un grafo.
2. Definir reglas para validar consultas y movimientos.
3. Implementar A* y compararlo con Dijkstra.
4. Evaluar rutas normales, cierres y errores de entrada.

Marco conceptual

La representación del conocimiento organiza los hechos que un programa necesita para razonar. En un sistema basado en reglas, las condiciones se evalúan sobre esos hechos para habilitar o impedir conclusiones y acciones (Benítez, 2014). En este proyecto los hechos son estaciones, troncales, tipos, posiciones y conexiones; las reglas determinan la validez de cada movimiento.

Una red de transporte puede representarse como un grafo ponderado. Los vértices corresponden a estaciones, las aristas a conexiones y los pesos al costo estimado. La búsqueda A* asigna a cada estado la evaluación $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado y $h(n)$ estima el costo restante. Si la heurística no sobreestima, A* conserva la optimalidad (Russell & Norvig, 2021).

Metodología y arquitectura

Módulo	Responsabilidad
base_conocimiento.py	25 estaciones, 26 conexiones, costos y posiciones.
reglas.py	Cinco reglas de validación e inferencia.
busqueda.py	A*, Dijkstra y reconstrucción de rutas.
main.py	Menú y presentación de resultados.
test_sistema.py	Siete pruebas automatizadas.

Base de conocimiento y reglas

R1: una estación es utilizable si existe y no está cerrada. R2: se puede viajar entre dos estaciones habilitadas si existe una conexión. R3: una estación declarada como transbordo permite cambiar de troncal. R4: una consulta exige estaciones existentes, abiertas y diferentes. R5: una ruta es válida si todos sus movimientos consecutivos cumplen R2.

Algoritmos

A* mantiene una frontera priorizada por el costo acumulado más la heurística. La heurística emplea la distancia euclidiana entre posiciones didácticas. El programa conserva el mejor costo conocido y el predecesor de cada nodo; al alcanzar el destino reconstruye la secuencia completa. Dijkstra usa la misma estructura sin heurística y actúa como oráculo de comparación.

Resultados de las pruebas

Caso	Condición	Resultado
1	Portal Norte a Calle 100	Ruta válida; 22 minutos; 8 nodos explorados.
2	Portal Norte a Portal El Dorado	A* y Dijkstra: mismo costo de 81 minutos.
3	Calle 76 cerrada	Ruta alternativa Heroes-Calle 72; 44 minutos.
4	Calle 26 cerrada	No existe ruta al Portal El Dorado.
5	Origen inexistente	Mensaje de error controlado.
6	Origen igual al destino	Consulta rechazada.
7	Regla de transbordo	Calle 26: verdadero; Marly: falso.

La ejecución de `python -m unittest discover -s tests -v` produjo siete pruebas correctas en Python. La comparación del caso extenso confirmó que A* y Dijkstra alcanzan el mismo costo. El cierre de Calle 76 activó una alternativa, mientras que el cierre de Calle 26 separó el destino del resto del grafo.

Limitaciones

El modelo no incluye horarios, congestión, frecuencia, sentidos operativos ni todos los servicios. Las coordenadas no son geográficas y los costos no son oficiales. Estas decisiones mantienen un alcance reproducible y permiten concentrarse en reglas y búsqueda.

Conclusiones

La integración de hechos, reglas y A* permitió obtener rutas explicables y reaccionar ante restricciones. Las reglas impiden que el algoritmo utilice estados inválidos; la heurística orienta la exploración; y Dijkstra ofrece una referencia independiente del costo mínimo. Las pruebas muestran que separar conocimiento, inferencia, búsqueda e interfaz mejora la verificabilidad del sistema.

Referencias

Benítez, R. (2014). *Inteligencia artificial avanzada*. Editorial UOC. Capítulos 2, 3 y 9.

Python Software Foundation. (2026). *Python 3 documentation*. <https://docs.python.org/3/>

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

TransMilenio S. A. (2026). *Sistema TransMilenio*. <https://www.transmilenio.gov.co/>

Repositorio

<https://github.com/AndresRamirez0/actividad-2-busqueda-transmilenio>